SECURITY OPERATIONS CENTER (SOC)

Investment Simulation Project

Project Title

SOC Investing Simulation Using Elastic Stack

Submitted By

Team Name: Cyber Defenders

Team Members:

1. **Ishita Kumari** – Team Leader
2. **Anshu Kumar**
3. **Anshu Yadav**
4. **Priya Parihar**

Technology Stack

- Ubuntu Linux
- macOS
- Suricata
- Elasticsearch
- Kibana
- Filebeat
- Apache Web Server
- KQL (Kibana Query Language)

Project Objective

To build a basic Security Operations Center (SOC) environment for collecting, monitoring, and analyzing system and web-server logs using the Elastic Stack.

DECLARATION

We hereby declare that this project titled "SOC Investing Simulation Using Elastic Stack" has been successfully completed as part of our internship training program.

The project demonstrates the implementation of a basic Security Operations Center (SOC) environment using **Suricata**,**Elasticsearch, Kibana, Filebeat, Apache Web Server, and KQL** for log collection, monitoring, and analysis.

Team Members:

Name.	Roll Number	Signature
Ishita Kumari	**2415000707**	
Anshu Kumar	**2415000270**	
Anshu Yadav	**2415000271**	
Priya Parihar	**2415001194**	

Date: _____

Place: _____

1. Introduction

A Security Operations Center (SOC) is responsible for monitoring, detecting, investigating, and responding to security-related events in an organization's IT infrastructure.

For this project, I created a small SOC monitoring environment using the **ELK Stack**. The objective was to collect web server and system logs from an Ubuntu server, send those logs to Elasticsearch using Suricata, Filebeat, and analyze them visually through Kibana.

The lab provides a practical environment for understanding how security logs are collected, centralized, searched, and investigated.

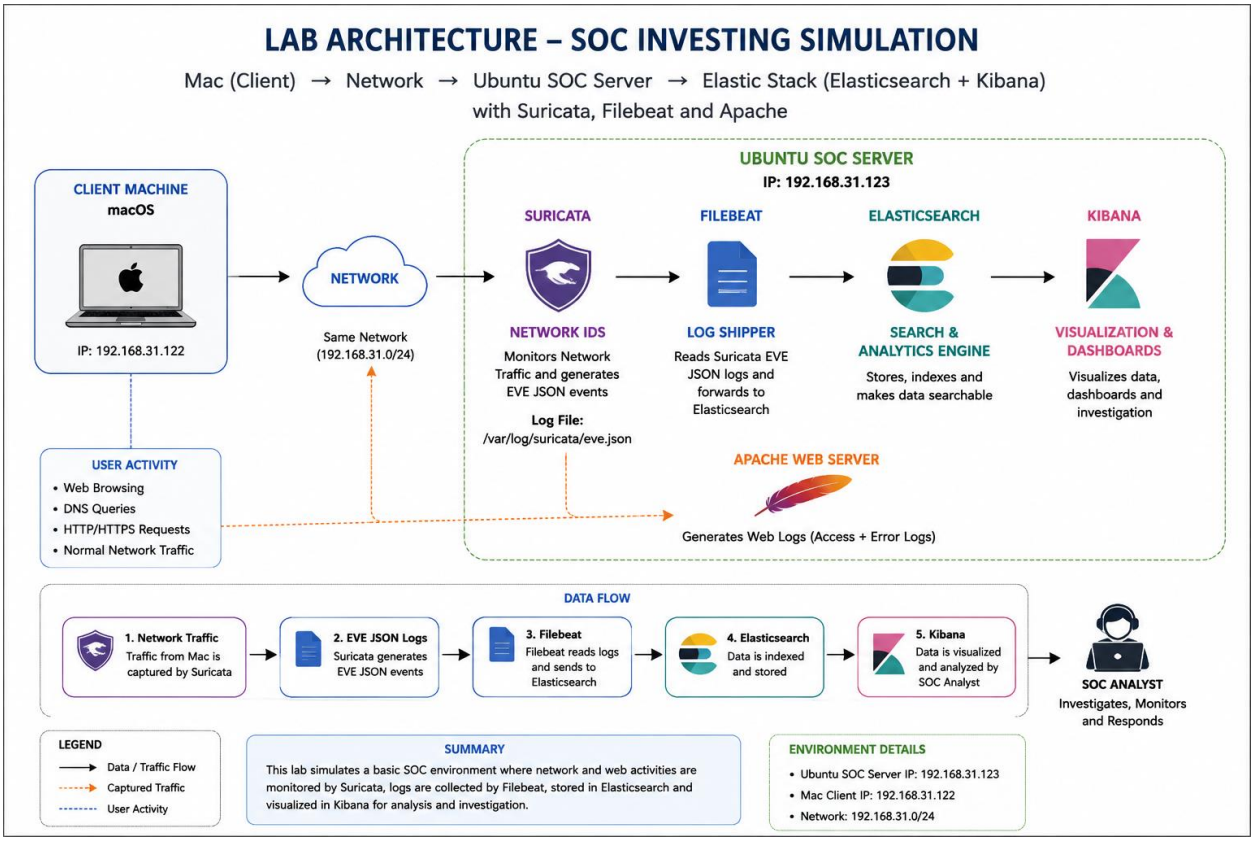
2. Project Objectives

The main objectives of this project were:

1. Set up an Ubuntu Server as the main monitoring server.
2. Install and configure Apache2.
3. Install and configure Suricata
4. Install and configure Elasticsearch.
5. Install and configure Kibana.
6. Install and configure Filebeat.
7. Collect Apache and system logs.
8. Send logs from Filebeat to Elasticsearch.
9. Create a Kibana Data View.
10. Analyze collected logs using Kibana Discover.
11. Generate web traffic from a MacBook.
12. Understand the complete log collection and monitoring workflow.

3. Lab Architecture & Data Flow

Our lab architecture and Data flow is:



MacBook

The MacBook was used as the client machine.

It was used for:

- Accessing Apache
- Accessing Kibana
- Generating HTTP requests
- Testing network connectivity

4. Lab Environment

Ubuntu Server

The Ubuntu Server was installed inside a virtual machine using UTM.

Operating System:

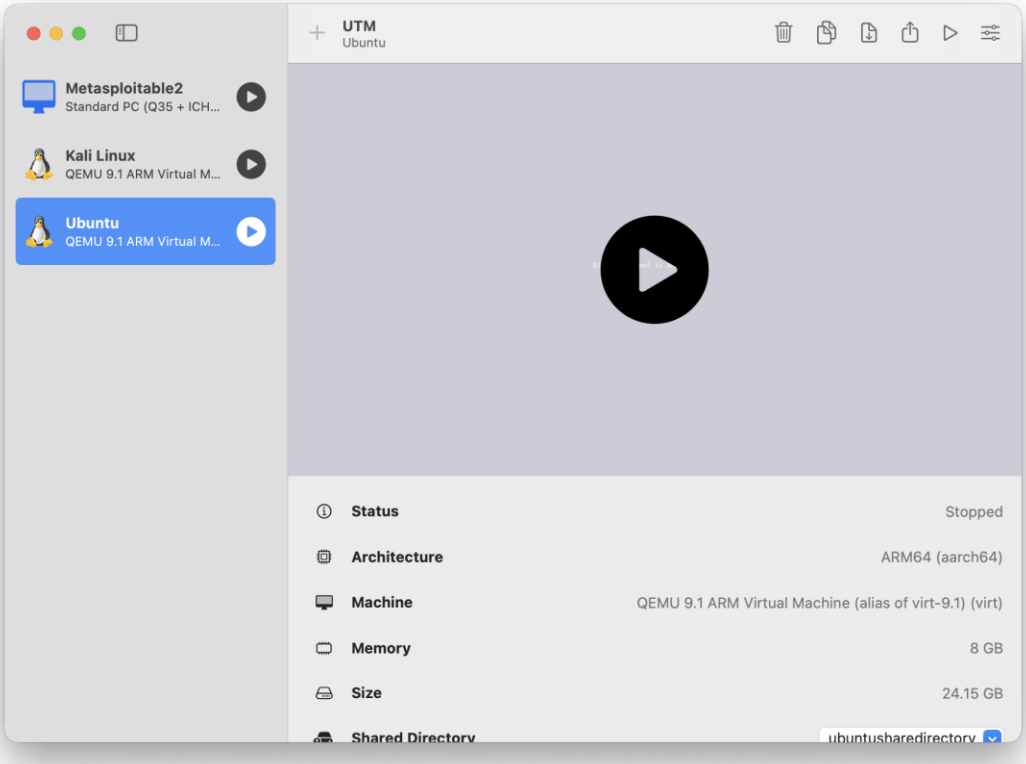
Ubuntu 24.04.4 LTS

Architecture:

ARM64 / AArch64

Ubuntu IP address used in the lab:

192.168.31.123



5. Checking Ubuntu IP Address

After starting Ubuntu, I checked its IP address using:

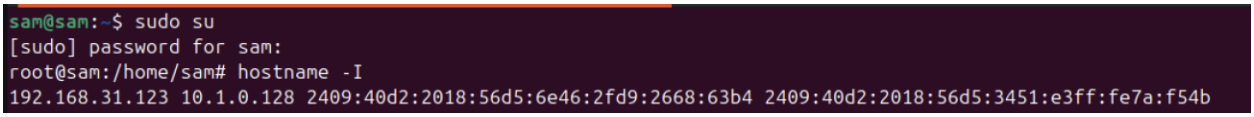
hostname -I

Example output:

192.168.31.123

This IP address was required to access Ubuntu services from the MacBook.

Screenshot



6. Testing Network Connectivity

From the MacBook, I tested whether Ubuntu was reachable:

ping 192.168.31.123

If successful, the output showed replies from the Ubuntu server.

Example:

64 bytes from 192.168.31.123: icmp_seq=0 ttl=64 time=...

This confirmed that the MacBook and Ubuntu server could communicate over the network.

7. Installing Apache2

Apache2 was installed as the web server.

Apache was selected because it generates useful access and error logs that can later be collected by Filebeat.

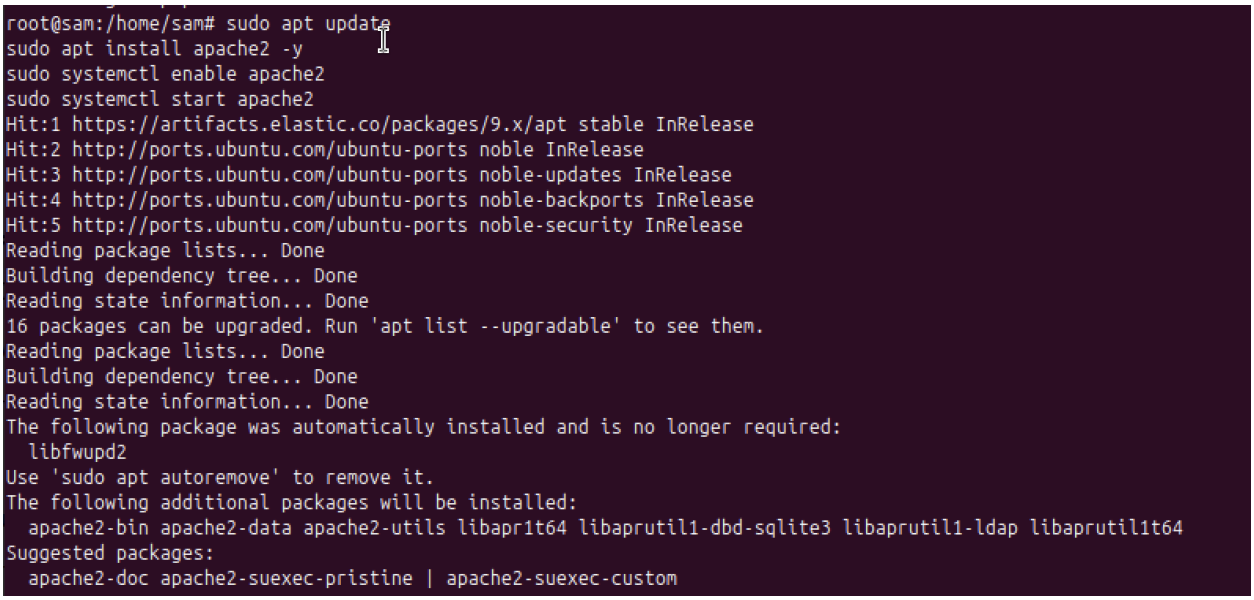
First, the package repository was updated:

sudo apt update

Apache2 was then installed:

sudo apt install apache2 -y
ache2

Screenshot



8. Starting Apache2

After installation, Apache2 was started:

sudo systemctl start apache2

To make Apache start automatically after reboot:

sudo systemctl enable apache2

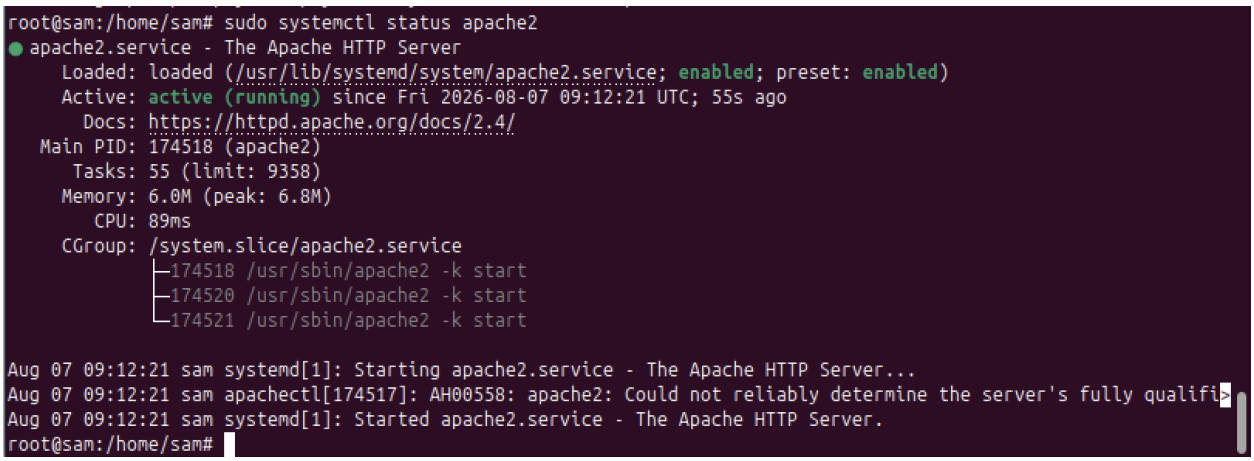
The service status was checked using:

sudo systemctl status apache2

Expected result:

Active: active (running)

Screenshot



apache2.service
Active: active (running)

9. Testing Apache from Ubuntu

Apache was tested locally using:

curl -I http://192.168.31.123

Expected output:

HTTP/1.1 200 OK
Server: Apache/2.4.58 (Ubuntu)
Content-Type: text/html

The 200 OK response confirmed that Apache was working correctly.

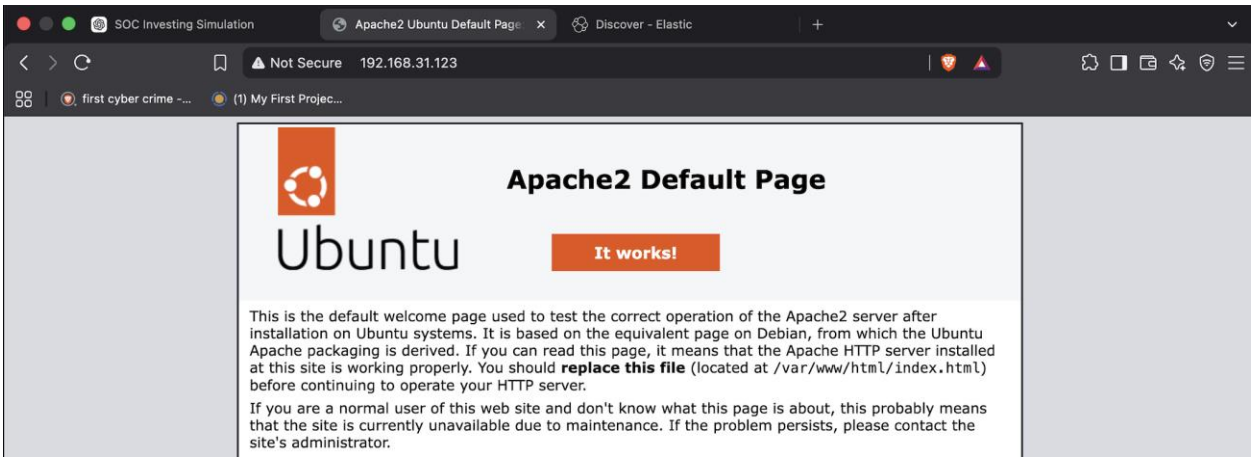
10. Testing Apache from MacBook

The Apache server was also accessed from the MacBook.In the browser:

http://192.168.31.123

The default Apache2 Ubuntu page appeared.This confirmed that the MacBook could access the Apache web server running inside Ubuntu.

Screenshot



11. Installing Elasticsearch

Elasticsearch was used as the central storage and search engine for the collected logs.

After configuring the Elastic package repository, Elasticsearch was installed.

Command:

sudo apt install elasticsearch

12. Starting Elasticsearch

Elasticsearch was started using:

sudo systemctl start elasticsearch

To start Elasticsearch automatically during system boot:

sudo systemctl enable elasticsearch

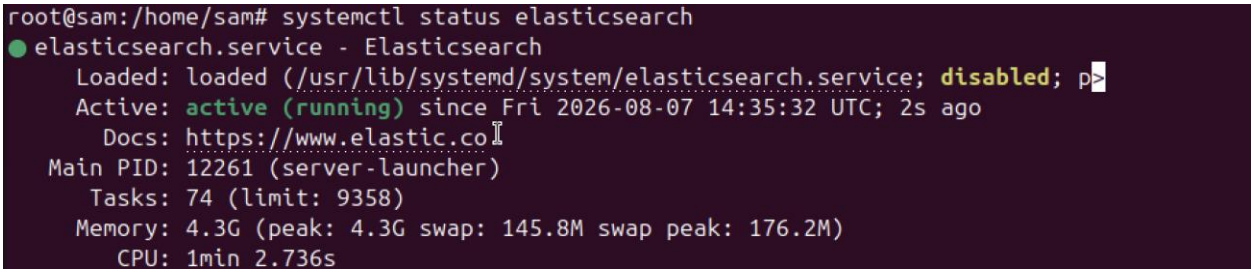
Its status was checked:

sudo systemctl status elasticsearch

Expected:

Active: active (running)

Screenshot



13. Testing Elasticsearch

Elasticsearch was tested using cURL:

curl -k -u elastic https://localhost:9200

The command requests the Elasticsearch API over HTTPS.

The -k option allows the connection even when certificate verification is not being performed in this lab environment.

The -u elastic option authenticates using the elastic user.

14. Checking Elasticsearch Indices

To check whether indexes were available:

curl -k -u elastic https://localhost:9200/_cat/indices?v

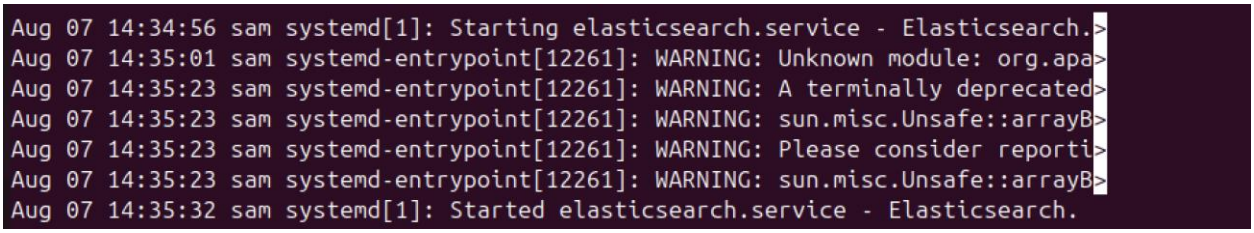
After Filebeat started sending data, an index similar to:

.ds-filebeat-9.5.0-2026.08.07-000001

appeared.

The document count increased as new logs were received.

Screenshot



15. Installing Kibana

Kibana was installed as the visualization and analysis interface.

Command:

sudo apt install kibana

Kibana provides a web interface through which Elasticsearch data can be searched and visualized.

16. Starting Kibana

Kibana was started using:

sudo systemctl start kibana

It was enabled at boot:

sudo systemctl enable kibana

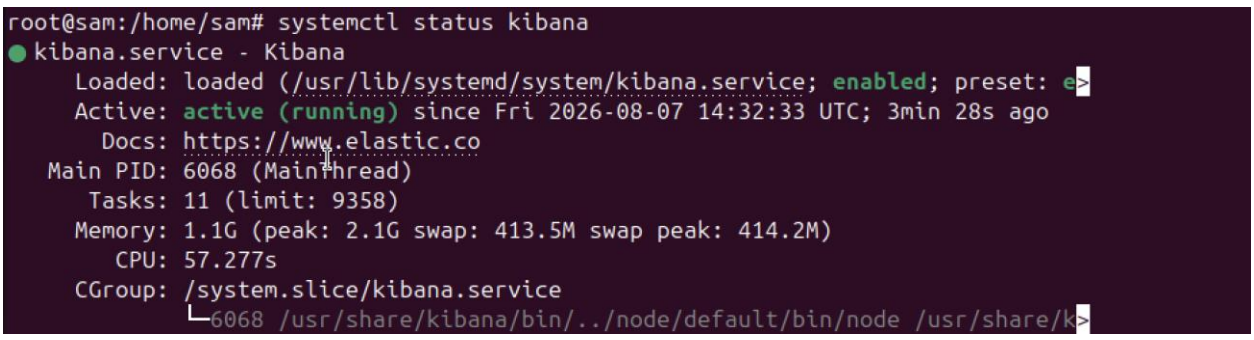
Status:

sudo systemctl status kibana

Expected:

Active: active (running)

Screenshot



17. Accessing Kibana from MacBook

Kibana was accessed from the MacBook browser using:

http://192.168.31.123:5601

Port 5601 is the default Kibana web interface port.

The Kibana login page appeared.

Screenshot

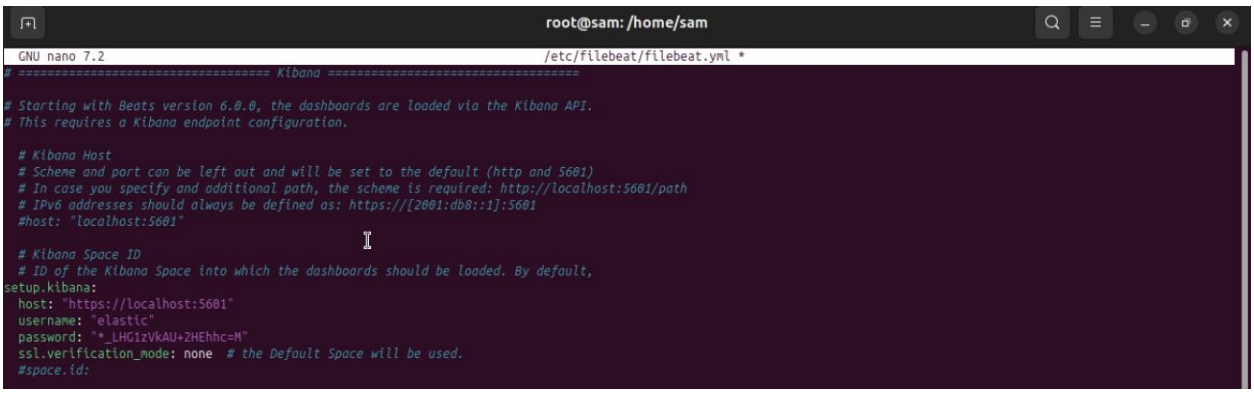
```
Aug 07 14:35:57 sam kibana[6068]: [2026-08-07T14:35:57.127+00:00][INFO ][plugin>
Aug 07 14:35:57 sam kibana[6068]: [2026-08-07T14:35:57.713+00:00][INFO ][plugin>
Aug 07 14:35:57 sam kibana[6068]: [2026-08-07T14:35:57.912+00:00][INFO ][plugin>
Aug 07 14:35:58 sam kibana[6068]: [2026-08-07T14:35:58.811+00:00][INFO ][plugin>
```

18. Kibana Login

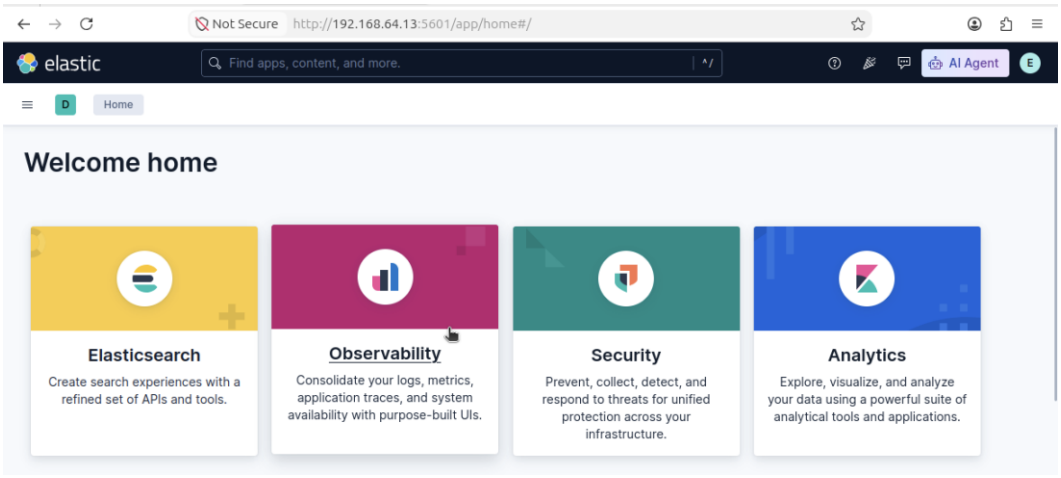
The Elastic account was used to log in.

Username: elastic

Password : *_LHG1zVkAU+2HEhhc=M



The password generated/configured during Elasticsearch setup was used.After successful authentication, the Kibana home interface was displayed.



19. Installing Filebeat

Filebeat is a lightweight log shipper developed by Elastic.

Its purpose in this project was to:

Read log files
↓
Process logs
↓
Send logs to Elasticsearch

Filebeat was installed using:

sudo apt install filebeat

20. Enabling Apache Module in Filebeat

Filebeat provides modules for different applications.

The Apache module was enabled:

sudo filebeat modules enable apache

The Apache module allows Filebeat to understand Apache access and error logs.

21. Enabling System Module

The system module was also enabled:

sudo filebeat modules enable system

This allows Filebeat to collect system-related logs such as:

- System logs
- Authentication logs
- Service-related events

22. Filebeat Configuration

The Apache module configuration was located at:

/etc/filebeat/modules.d/apache.yml

It contained configuration for:

apache access logs
apache error logs

The relevant configuration was:

```
- module: apache
  access:
    enabled: true


  error:
    enabled: true
```

23. Apache Log Locations

The main Apache log files on Ubuntu were:

/var/log/apache2/access.log
/var/log/apache2/error.log

Access Log

The access log records incoming HTTP requests.

For example:

GET /
GET /favicon.ico

Error Log

The error log records errors and warnings generated by Apache.

24. Filebeat Setup

Filebeat setup was executed using:

sudo filebeat setup

This command prepares required assets such as:

- Index templates
- Ingest pipelines
- Kibana assets

During setup, Elasticsearch ingest pipelines were loaded.

25. Filebeat Configuration Error

During the configuration process, Filebeat initially failed.

One error was:

module apache is configured but has no enabled filesets

This occurred because the module configuration was not correctly structured.

The Apache configuration file was checked:

cat /etc/filebeat/modules.d/apache.yml

The YAML indentation was corrected so that enabled: true was properly nested under the corresponding fileset.

26. Another Filebeat Error

A later test showed:

module system is configured but has no enabled filesets

This indicated that the system module also needed its filesets correctly enabled.

After correcting the configuration, Filebeat was tested again.

27. Testing Filebeat Manually

Filebeat was run in the foreground:

sudo filebeat -e

This helped identify configuration problems directly from the terminal.

After fixing the module configuration, Filebeat successfully initialized.

Important output included:

filebeat start running

and:

Enabled modules/filesets: apache (access), apache (error)

This confirmed that the Apache filesets were enabled.

28. Starting Filebeat as a Service

After fixing the configuration:

sudo systemctl restart filebeat

Then:

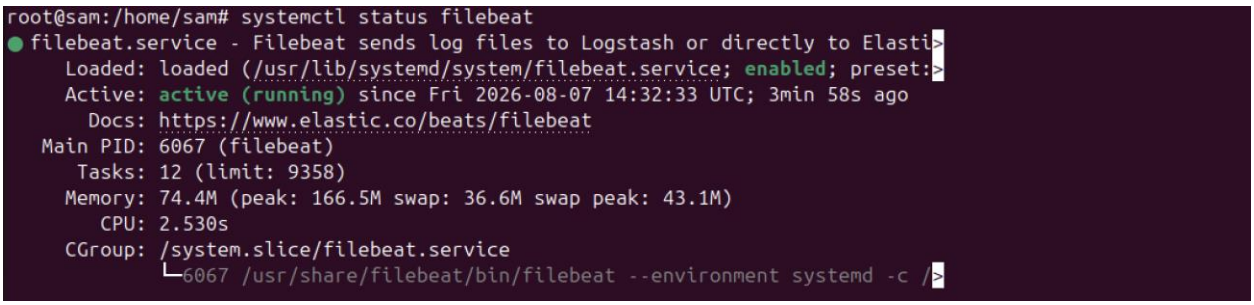
sudo systemctl status filebeat

Expected:

Active: active (running)

This was an important milestone because Filebeat was now continuously collecting logs.

Screenshot



Active: active (running)

29. Verifying Filebeat Data in Elasticsearch

The Elasticsearch indexes were checked again:

curl -k -u elastic https://localhost:9200/_cat/indices?v

The Filebeat data stream appeared:

.ds-filebeat-9.5.0-2026.08.07-000001

The docs.count value increased whenever new logs were collected.

This confirmed the following flow:

Apache Logs
↓
Filebeat
↓
Elasticsearch

30. Creating Kibana Data View

After Filebeat started sending data, Kibana was opened from the MacBook.

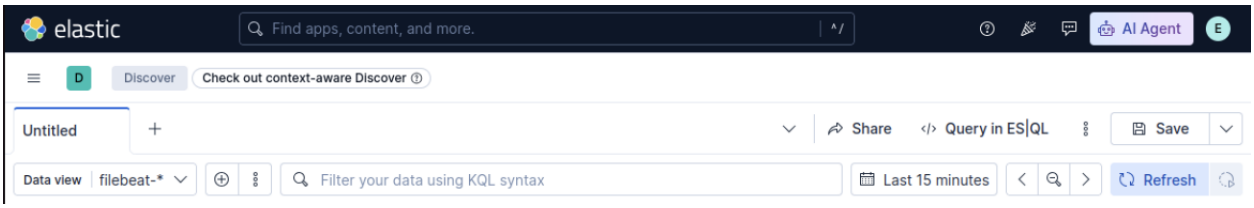
Navigate to:

Discover

The Data View was set to:

filebeat-*

The wildcard * allows Kibana to search Filebeat indexes matching the pattern.



31. Using Kibana Discover

The Discover page was opened with:

Data View: filebeat-*

Kibana displayed:

- Event timestamps
- Agent information
- Host information
- Apache events
- System events
- Other collected fields

The histogram showed the number of events over time.

32. KQL Filtering

Kibana supports **KQL (Kibana Query Language)** for searching and filtering logs.

The KQL search box is located at the top of Discover.

For Apache access logs:

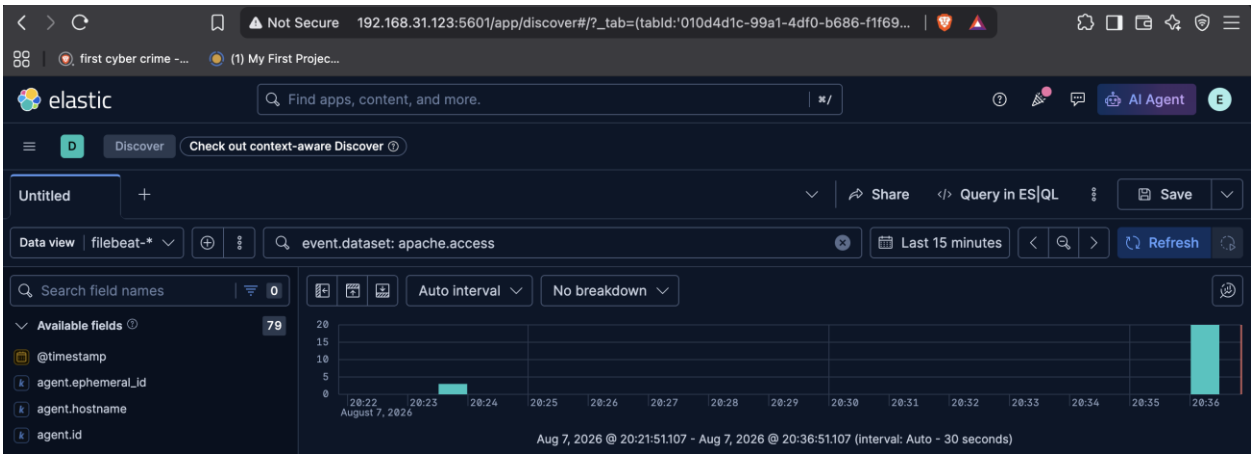
event.dataset: apache.access

For Apache error logs:

event.dataset: apache.error

For Apache-related events:

event.dataset: apache.*



33. Generating New Apache Logs

To generate new HTTP requests, the MacBook was used.

From the Mac Terminal: curl http://192.168.31.123

Multiple requests can also be generated: for i in {1..20}; do curl http://192.168.31.123; done

Each request generates an Apache access-log event.

34. Observing Logs in Kibana

After generating requests, Kibana Discover was refreshed.

The new Apache events appeared in Discover.

This demonstrated the complete real-time log pipeline:

Mac curl request

↓

Apache

↓

access.log

↓

Filebeat

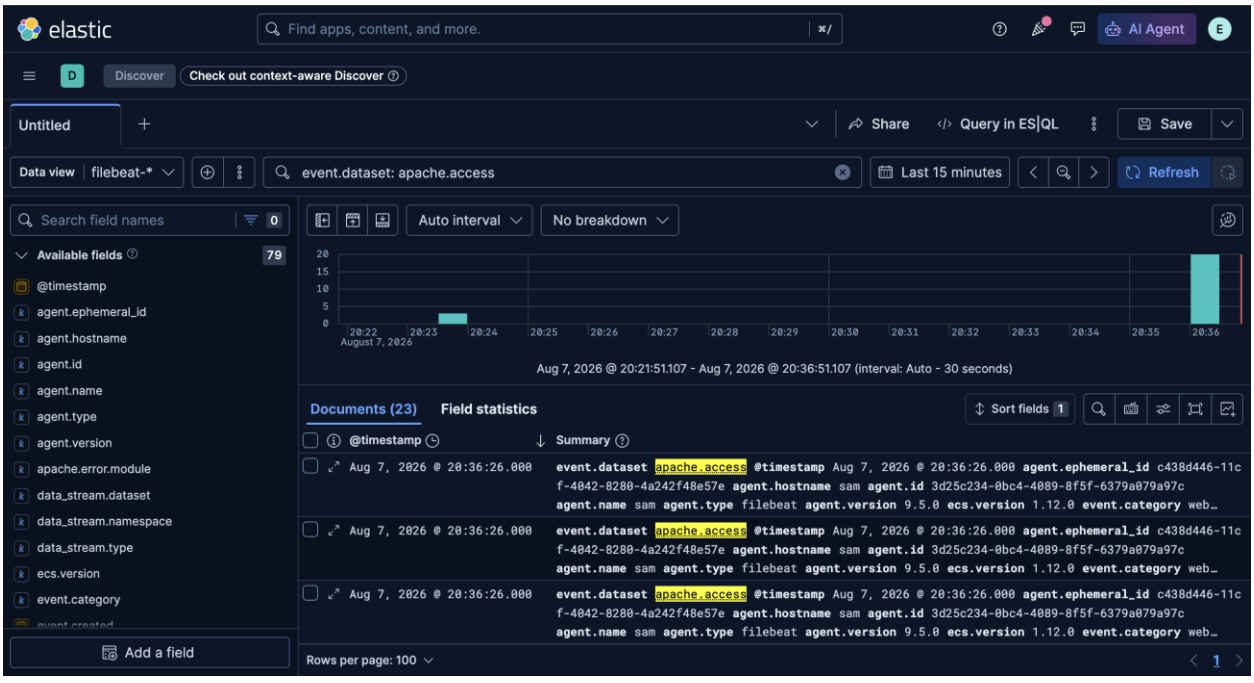
↓

Elasticsearch

↓

Kibana Discover

Screenshot



35. Useful Fields for SOC Investigation

Important fields that can be investigated include:

source.ip
http.request.method
url.path
http.response.status_code
user_agent.original
host.name
@timestamp

These fields help an analyst understand:

- Who made the request?
- What URL was requested?
- Which HTTP method was used?
- What response did the server return?
- Which user agent was used?
- When did the event occur?

36. Investigating HTTP Errors

Kibana can be used to search for HTTP errors.

For example:

http.response.status_code: 404

This searches for requests where the server returned a 404 Not Found response.

Another example:

http.response.status_code: 500

This searches for server-side errors.

37. Network Troubleshooting

During the project, connectivity between the MacBook and Ubuntu VM was tested multiple times.

Ubuntu IP:

192.168.31.123

MacBook and Ubuntu needed to be on a network path that allowed communication.

Connectivity was tested with:

ping 192.168.31.123

Apache was tested using:

curl http://192.168.31.123

The Ubuntu routing table was checked using:

ip route

Network interfaces were checked using:

ip addr

Neighbor information was checked using:

ip neigh

38. Checking UFW

Ubuntu's firewall status was checked using:

sudo ufw status verbose

In our lab, the result was:

Status: inactive

Therefore, UFW was not blocking the Apache connection.

39. Checking Apache Port

To verify that Apache was listening on port 80:

sudo ss -tlnp | grep :80

The output showed:

LISTEN ... :80 ... apache2

This confirmed that Apache was listening for HTTP connections.

40. Installing Suricata

Suricata was added to the SOC lab as a Network Intrusion Detection System (NIDS). It monitors network traffic and generates security events when suspicious or malicious activity is detected.

Step 1: Update Ubuntu Packages

sudo apt update

Screenshot: Terminal showing successful package update.

Step 2: Install Suricata

sudo apt install suricata -y

Screenshot: Suricata installation completed successfully.

Step 3: Check Suricata Version

suricata --build-info or suricata –V

Screenshot:

```
root@sam:/home/sam# suricata -v
Suricata 7.0.3
USAGE: suricata [OPTIONS] [BPF FILTER]

        -c <path>                           : path to configuration file
        -T                                  : test configuration file (use with
 -c)
        -i <dev or ip>                      : run in pcap live mode
        -F <bpf filter file>                : bpf filter file
        -r <path>                           : run in pcap file/offline mode
        -q <qid[:qid]>                      : run in inline nfqueue mode (use c
olon to specify a range of queues)
        -s <path>                           : path to signature file loaded in
addition to suricata.yaml settings (optional)
        -S <path>                           : path to signature file loaded exc
lusively (optional)
        -l <dir>                            : default log directory
        -D                                  : run as daemon
        -k [all|none]                       : force checksum check (all) or dis
abled it (none)
        -V                                  : display Suricata version
        -v                                  : be more verbose (use multiple tim
```

41. Starting and Enabling Suricata

Start the Suricata service:

sudo systemctl start suricata

Enable Suricata to start automatically when Ubuntu boots:

sudo systemctl enable suricata

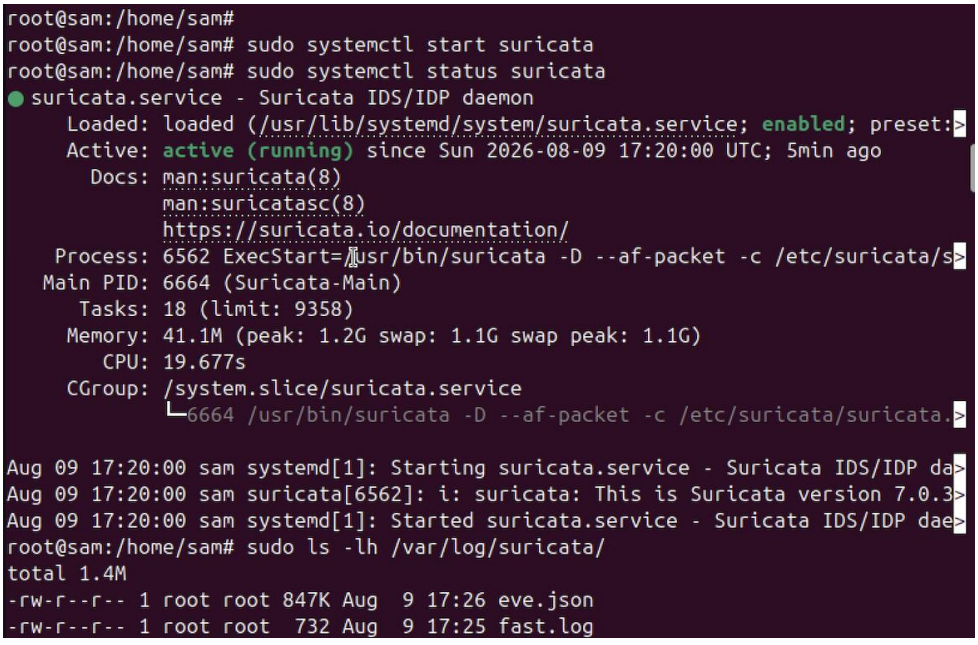
Check the service status:

sudo systemctl status suricata

The expected result is:

Active: active (running)

Screenshot:



service status showing active (running).

42. Configuring Suricata

The main Suricata configuration file is:

/etc/suricata/suricata.yaml

Open the configuration file:

sudo nano /etc/suricata/suricata.yaml

The network interface used by the Ubuntu system was identified using:

ip addr

In our lab, the main interface was:

enp0s1

The internal network was configured according to the lab network.

After making the required configuration changes, restart Suricata:

sudo systemctl restart suricata

Verify the configuration:

sudo suricata -T -c /etc/suricata/suricata.yaml

A successful configuration test confirms that Suricata can start with the current configuration.

Screenshot:

```
root@sam:/home/sam# sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode
i: suricata: Configuration provided was successfully loaded. Exiting.
root@sam:/home/sam#
```

Successful Suricata configuration test.

43. Suricata EVE JSON Logs

Suricata generates security and network events in EVE JSON format.

The main log file used in our project is:

/var/log/suricata/eve.json

Check whether Suricata is generating events:

sudo tail -f /var/log/suricata/eve.json

When network activity occurs, Suricata generates JSON-based events containing information such as:

- **Source IP**
- **Destination IP**
- **Source port**
- **Destination port**
- **Protocol**
- **HTTP information**
- **DNS information**
- **Alert information**
- **Timestamp**
- **Network flow information**

Screenshot: eve.json showing generated events.

44. Connecting Suricata with Filebeat

Filebeat was used to collect Suricata's EVE JSON logs and send them to Elasticsearch.

First, check the available Filebeat modules:

sudo filebeat modules list

Enable the Suricata module:

sudo filebeat modules enable suricata

Check that the module is enabled:

sudo filebeat modules list

The Suricata module should appear under the Enabled modules section.

45. Configuring the Filebeat Suricata Module

The Suricata module configuration file is:

/etc/filebeat/modules.d/suricata.yml

Open it:

sudo nano /etc/filebeat/modules.d/suricata.yml

Make sure the Suricata module is enabled and configured to read:

/var/log/suricata/eve.json

Save the configuration.

46. Restarting Filebeat

After configuring the Suricata module, restart Filebeat:

sudo systemctl restart filebeat

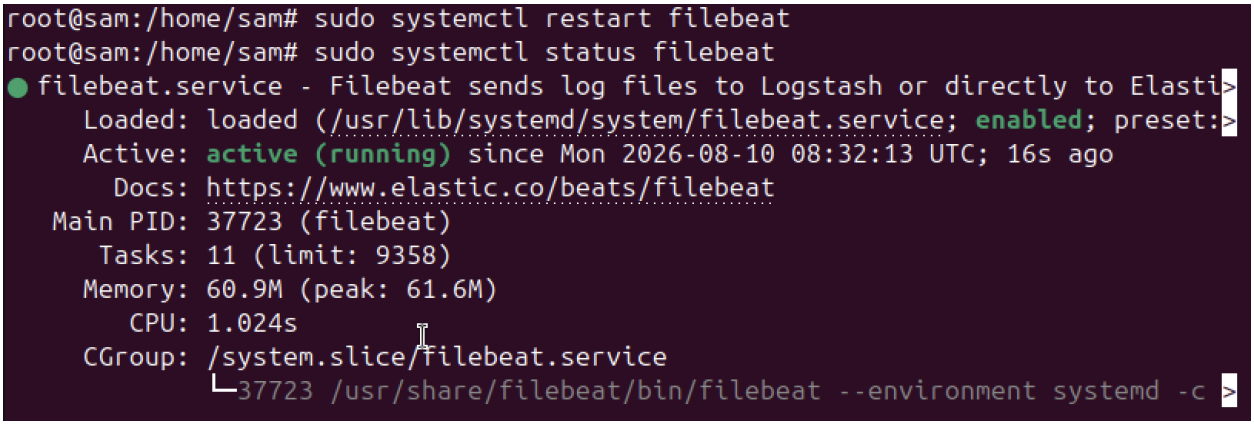
Check Filebeat status:

sudo systemctl status filebeat

Expected result:

Active: active (running)

Screenshot:



Filebeat service running successfully.

47. Verifying Suricata Data in Elasticsearch

Check the Elasticsearch indices:

curl -k -u elastic:'PASSWORD' https://localhost:9200/_cat/indices?v

The Filebeat data stream should be visible, for example:

.ds-filebeat-...

This confirms that Filebeat is successfully sending collected data to Elasticsearch.

48. Viewing Suricata Data in Kibana

Open Kibana from the Mac browser using the Ubuntu server IP:

http://192.168.31.123:5601

Log in using the Kibana/Elastic credentials.

Go to:

Kibana → Discover

Select the data view:

filebeat-*

Use the KQL search bar to filter Suricata events:

event.module : "suricata"

You can also use:

event.dataset : "suricata.eve"

This displays events generated by Suricata.

49. Testing Network Activity

To generate network traffic, use the MacBook as the client machine.

For example, open a website or generate normal network activity from the Mac.

Suricata monitors the traffic reaching the configured interface and generates events.

The flow is:

MacBook
↓
Network Traffic
↓
Suricata
↓
/var/log/suricata/eve.json
↓
Filebeat
↓
Elasticsearch
↓
Kibana

↓

SOC Analyst

50. Filtering Suricata Events Using KQL

In Kibana Discover, use:

event.module : "suricata"

This filters the data and displays only Suricata-related events.

Useful fields that can be examined include:

source.ip
destination.ip
source.port
destination.port
network.protocol
http.request.method
http.response.status_code
url.path
user_agent.original

These fields help the SOC analyst understand who communicated with whom, which protocol was used, what type of request occurred, and whether Suricata generated an alert.

51. Suricata Dashboard

Kibana may also provide pre-built Filebeat Suricata dashboards after the Suricata integration is installed.

Navigate to:

Kibana → Dashboards

Look for dashboards such as:

- **[Filebeat Suricata] Events Overview**
- **[Filebeat Suricata] Alert Overview**

These dashboards provide a visual overview of Suricata events and alerts.

Screenshot:

52. Complete Startup Procedure

After restarting Ubuntu, the services can be started using:

```
sudo systemctl start elasticsearch
sudo systemctl start suricata
sudo systemctl start kibana
sudo systemctl start filebeat
sudo systemctl start apache2
```

Or restarted using:

```
sudo systemctl restart elasticsearch
sudo systemctl restart suricata
sudo systemctl restart kibana
sudo systemctl restart filebeat
sudo systemctl restart apache2
```

Then verify:

```
sudo systemctl status elasticsearch
sudo systemctl status kibana
sudo systemctl status suricata
sudo systemctl status filebeat
sudo systemctl status apache2
```

53. What I Learned

- **Through this project, I learned:**
- SOC & SIEM fundamentals
- Linux service management
- Apache log generation
- Suricata IDS & network monitoring
- EVE JSON event analysis
- Filebeat log collection
- Elasticsearch indexing & storage
- Kibana visualization
- KQL-based log filtering
- HTTP & network traffic analysis
- Security-event investigation
- End-to-end **Log Source → Filebeat → Elasticsearch → Kibana** workflow

54. Conclusion

This project successfully demonstrated the implementation of a **basic Security Operations Center (SOC) monitoring environment** using the Elastic Stack, Filebeat, Apache, and Suricata.

The project established an end-to-end security monitoring workflow in which **network and system activities are generated, monitored, collected, stored, and analyzed**. Suricata was used for network traffic monitoring and security-event generation, while Filebeat collected the logs and forwarded them to Elasticsearch. Kibana was then used to visualize, search, and investigate the collected events using KQL.

Overall, this project provided practical experience in **log management, network monitoring, SIEM concepts, security-event analysis, and SOC operations**, while demonstrating how different security tools can work together to provide centralized visibility into an environment.

55. Appendix A — Important Commands

System Information

hostname -I
ip addr
ip route
ip neigh

Apache

sudo apt update
sudo apt install apache2 -y
sudo systemctl start apache2
sudo systemctl enable apache2
sudo systemctl status apache2

Elasticsearch

sudo systemctl start elasticsearch
sudo systemctl enable elasticsearch
sudo systemctl status elasticsearch
curl -k -u elastic https://localhost:9200
curl -k -u elastic https://localhost:9200/_cat/indices?v

Kibana

sudo systemctl start kibana
sudo systemctl enable kibana
sudo systemctl status kibana

Filebeat

sudo apt install filebeat
sudo filebeat modules enable apache
sudo filebeat modules enable system
sudo filebeat setup

sudo filebeat -e
sudo systemctl restart filebeat
sudo systemctl status filebeat

Suricata

sudo apt update
sudo apt install suricata -y

sudo systemctl start suricata
sudo systemctl enable suricata
sudo systemctl status suricata
sudo suricata --build-info
sudo suricata -T -c /etc/suricata/suricata.yaml
sudo tail -f /var/log/suricata/eve.json

Suricata → Filebeat Connection

sudo nano /etc/filebeat/filebeat.yml
sudo filebeat test config
sudo filebeat test output
sudo systemctl restart filebeat
sudo systemctl enable filebeat
sudo systemctl status filebeat

Network Testing

ping 192.168.31.123
curl http://192.168.31.123
curl -I http://192.168.31.123
sudo ss -tlnp | grep :80
sudo ufw status verbose